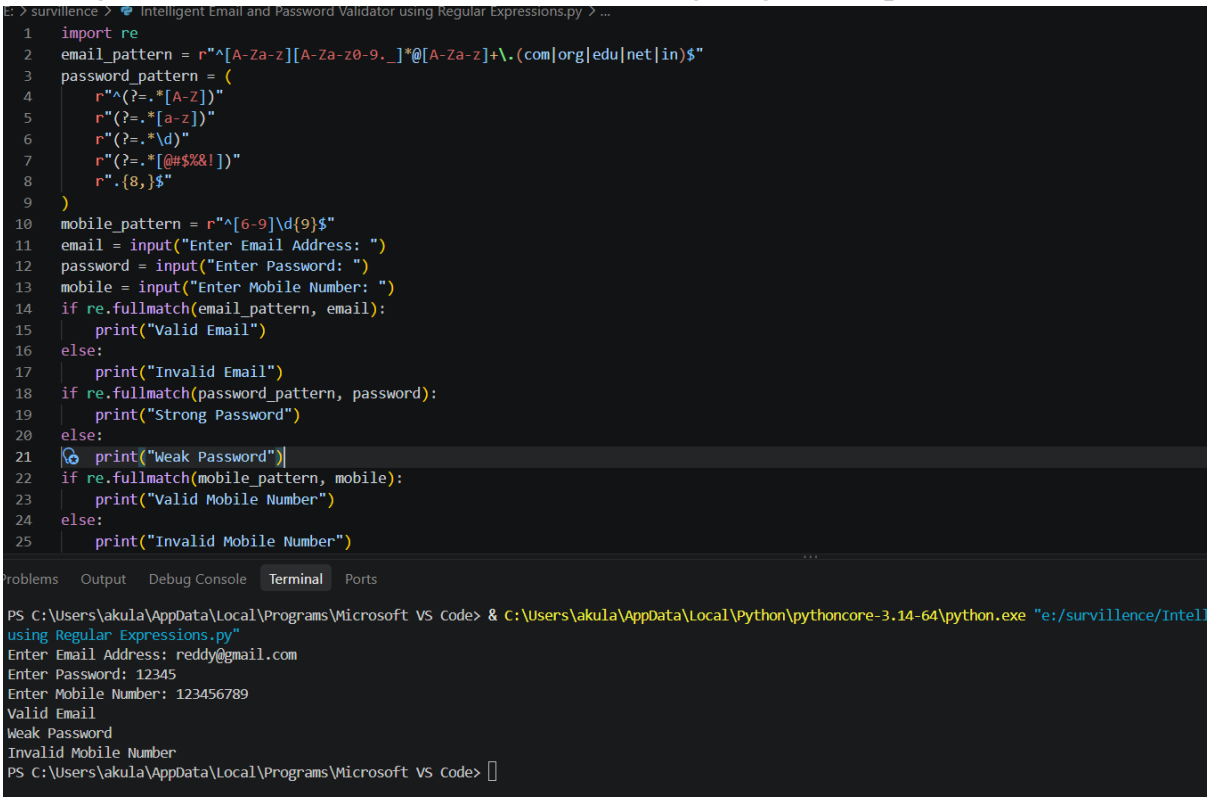
1. Intelligent Email and Password Validator using Regular Expressions

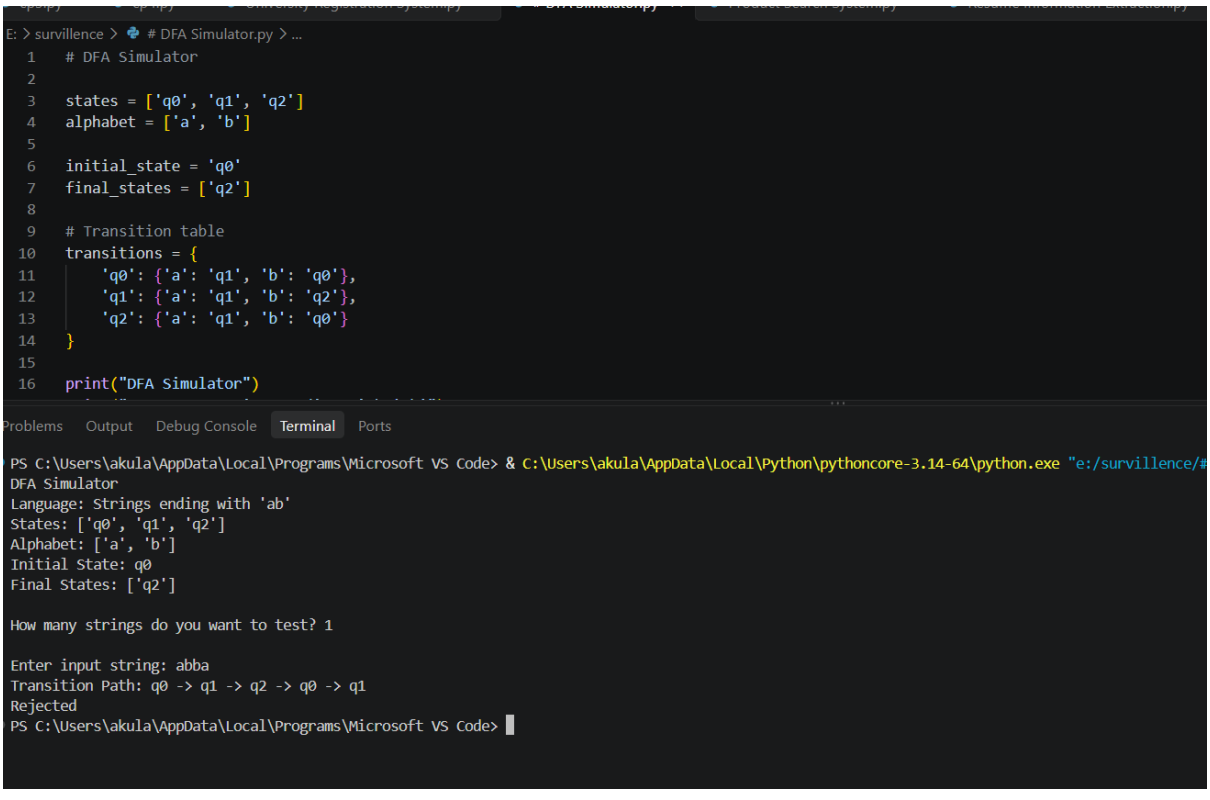


```
E: > survillence > Intelligent Email and Password Validator using Regular Expressions.py > ...
1   import re
2   email_pattern = r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$"
3   password_pattern = (
4       r"^(?=.*[A-Z])"
5       r"(?=.*[a-z])"
6       r"(?=.*\d)"
7       r"(?=.*[@#$%&!])"
8       r".{8,}$"
9   )
10  mobile_pattern = r"^[6-9]\d{9}$"
11  email = input("Enter Email Address: ")
12  password = input("Enter Password: ")
13  mobile = input("Enter Mobile Number: ")
14  if re.fullmatch(email_pattern, email):
15      print("Valid Email")
16  else:
17      print("Invalid Email")
18  if re.fullmatch(password_pattern, password):
19      print("Strong Password")
20  else:
21      print("Weak Password")
22  if re.fullmatch(mobile_pattern, mobile):
23      print("Valid Mobile Number")
24  else:
25      print("Invalid Mobile Number")

Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\akula\AppData\Local\Programs\Microsoft VS Code> & C:\Users\akula\AppData\Local\Python\pythoncore-3.14-64\python.exe "e:/survillence/Intell
using Regular Expressions.py"
Enter Email Address: reddy@gmail.com
Enter Password: 12345
Enter Mobile Number: 123456789
Valid Email
Weak Password
Invalid Mobile Number
PS C:\Users\akula\AppData\Local\Programs\Microsoft VS Code>
```

2. Design a Finite State Automata (FSA) Simulator



```
E: > survillence > # DFA Simulator.py > ...
1   # DFA Simulator
2
3   states = ['q0', 'q1', 'q2']
4   alphabet = ['a', 'b']
5
6   initial_state = 'q0'
7   final_states = ['q2']
8
9   # Transition table
10  transitions = {
11      'q0': {'a': 'q1', 'b': 'q0'},
12      'q1': {'a': 'q1', 'b': 'q2'},
13      'q2': {'a': 'q1', 'b': 'q0'}
14  }
15
16  print("DFA Simulator")

Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\akula\AppData\Local\Programs\Microsoft VS Code> & C:\Users\akula\AppData\Local\Python\pythoncore-3.14-64\python.exe "e:/survillence/#
DFA Simulator
Language: Strings ending with 'ab'
States: ['q0', 'q1', 'q2']
Alphabet: ['a', 'b']
Initial State: q0
Final States: ['q2']

How many strings do you want to test? 1

Enter input string: abba
Transition Path: q0 -> q1 -> q2 -> q0 -> q1
Rejected
PS C:\Users\akula\AppData\Local\Programs\Microsoft VS Code>
```

3. Smart Pattern Matching Engine using Regular Expressions

